NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007/CE1007/CZ1007: Data Structures & Algorithm

# Binary Trees



Presented by Dr Owen Noel Newton Fernando

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



# Objectives

- Introduce tree structures and key tree terminology.
- Explain the properties and organisation of binary trees.
- Demonstrate how binary tree nodes are represented and linked.
- Explain pre-order, in-order, and post-order traversals.
- Introduce level-order traversal using a queue.
- Demonstrate recursive and iterative solutions to binary tree problems.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Learning Outcome



By the end of this lecture, students should be able to:

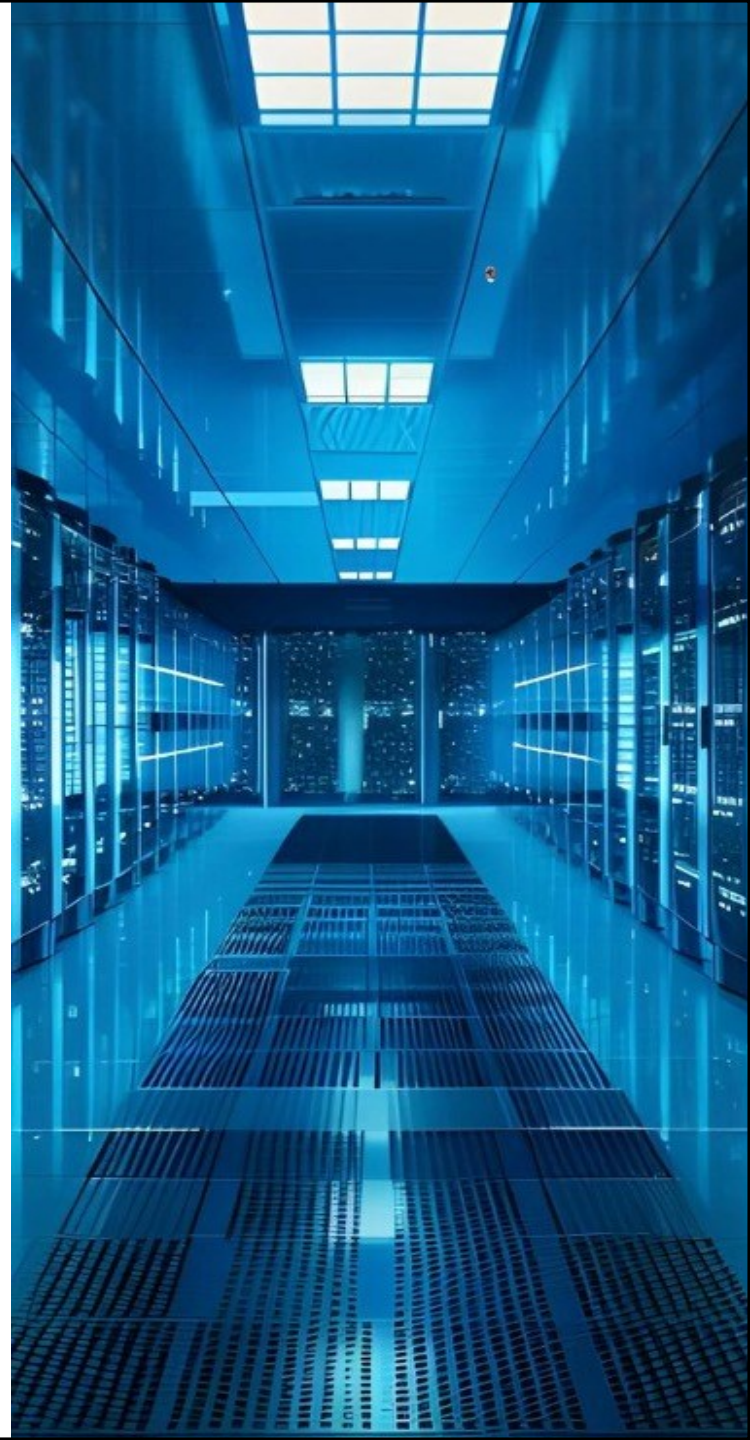
- Identify and explain the main components of a tree structure.
- Describe and represent the structure of a binary tree.
- Trace and implement pre-order, in-order, and post-order traversals.
- Apply recursion to solve common binary tree problems.
- Perform and implement level-order traversal using a queue.
- Use a stack to perform iterative pre-order traversal.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Contents


- Introduction to Trees and Non-Linear Data Structures
- Tree Terminology and Structure
- Binary Trees and Node Representation
- Depth-First Traversal
  - Pre-order Traversal
  - In-order Traversal
  - Post-order Traversal
- Breadth-First / Level-Order Traversal
- Binary Tree Applications
  - Counting Nodes
  - Finding Kth-Level Descendants
  - Calculating Tree Height
- Pre-order Traversal Using a Stack

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Introduction to Trees and Non-Linear Data Structures

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

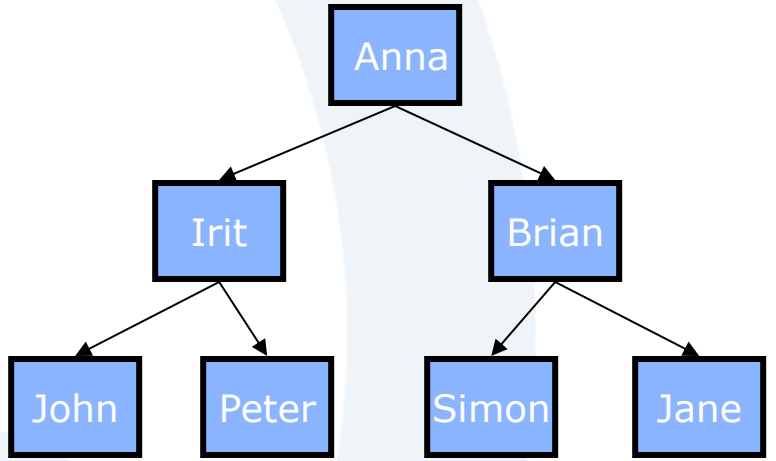
# From Linear to Non-Linear Data Structures

- **Linear Data Structures**
  - Elements are organised in a sequential order.
  - Each element follows a single logical path.
  - Examples: Linked Lists, Stacks, Queues, Priority Queues.
- **Non-Linear Data Structures**
  - Elements are organised in hierarchical or interconnected relationships.
  - An element may be connected to multiple other elements.
  - Trees are a fundamental non-linear data structure.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# What is a Tree?

- A **tree** is a hierarchical data structure composed of **nodes and edges**.
- A tree starts with a special node called the **root**.
- The **root has no parent**.
- Every other node has **exactly one parent**.
- A node may have **one or more children**.
- Nodes are connected through **parent–child relationships**.




Tree

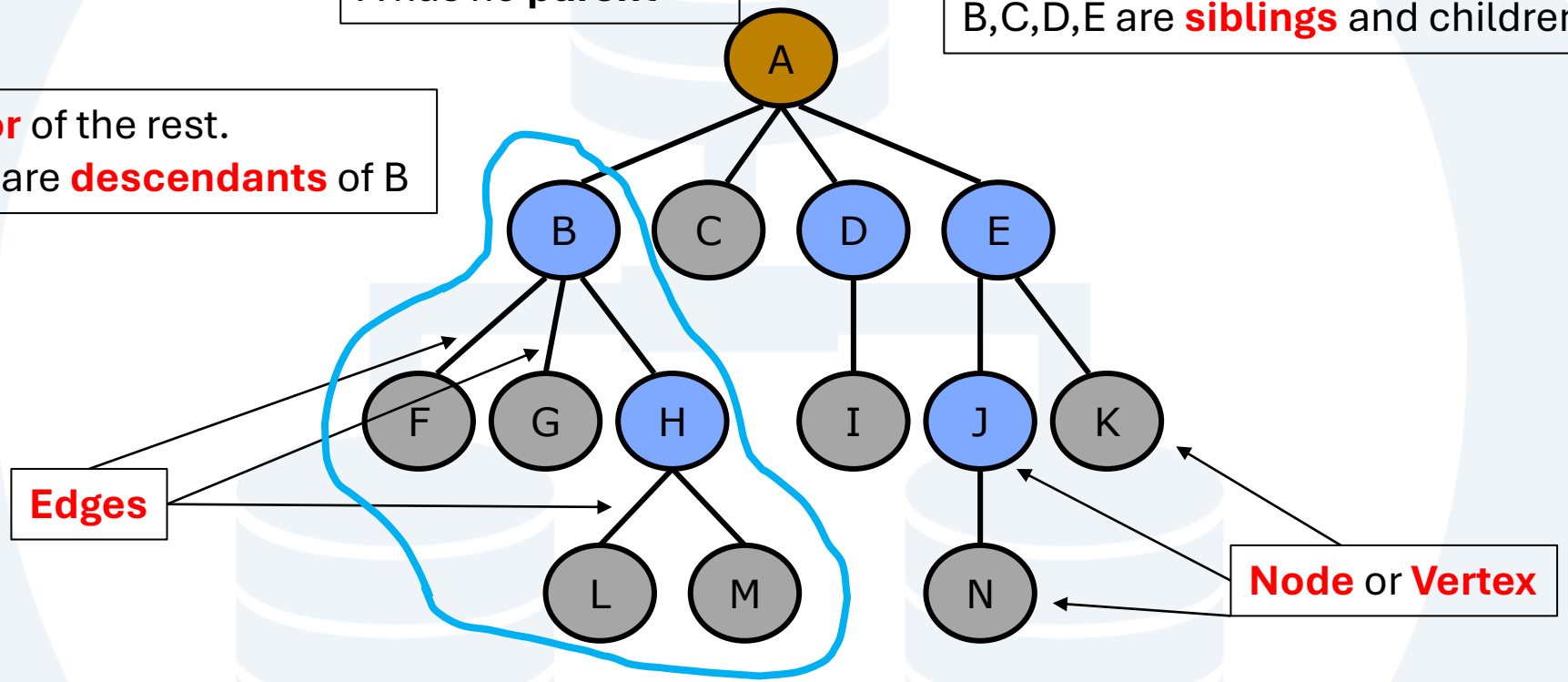
© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Understanding Tree Relationships

A is the **root** of Tree
A has no **parent**

A is the **parent** of B,C,D,E
B,C,D,E are **siblings** and children of A

A is **ancestor** of the rest.
F, G, H, L, M are **descendants** of B

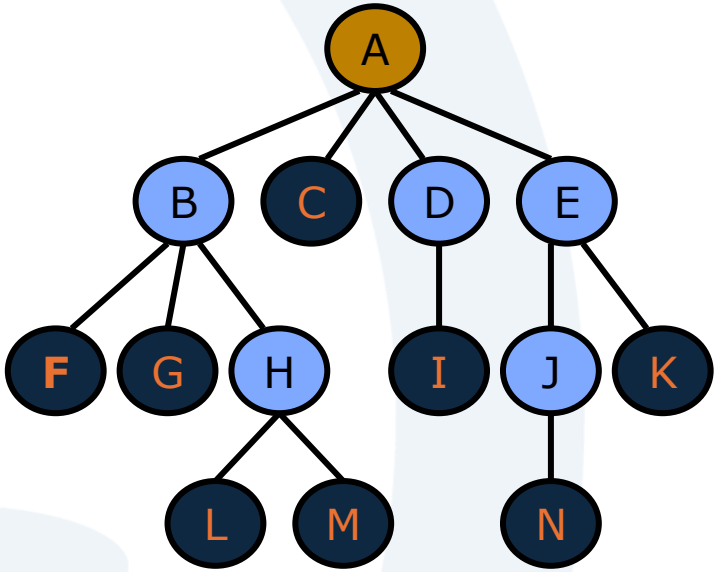



L and M have no **children**
L and M are **leaves**

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Terminology in Tree Structure

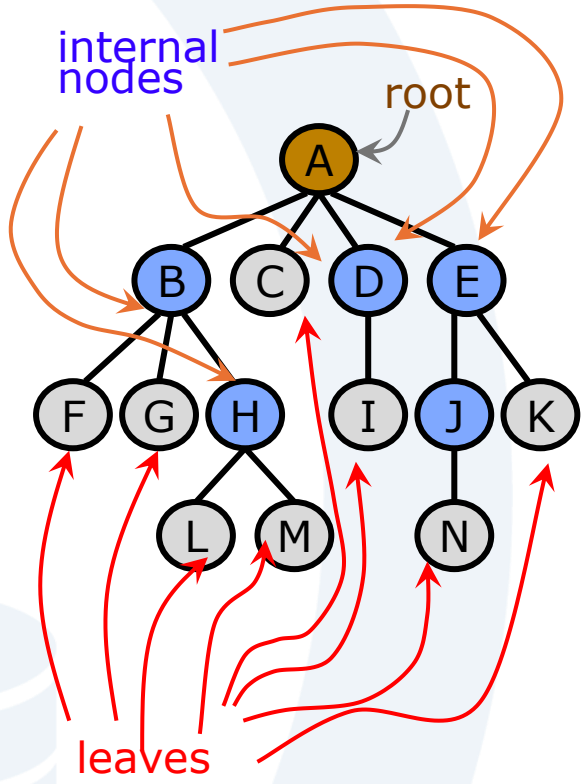
- **Root**: The topmost node of the tree.
- **Parent**: A node that has one or more children.
- **Child**: A node directly connected below its parent.
- **Siblings**: Nodes that share the same parent.
- **Ancestor**: A node on the path towards the root.
- **Descendant**: A node below another node in the hierarchy.
- Example: If A has children B, C, D and E, then A is their parent, while B, C, D and E are siblings.




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Terminology in Tree Structure (cont.)

- A tree is composed of nodes
- Types of nodes
  - Root: only one in a tree, has no parent.
  - Internal node(non-leaf):
    Nodes with children are called internal nodes
  - Leaf (External Node):
    nodes without children are called leaves




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Introduction to Binary Trees

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

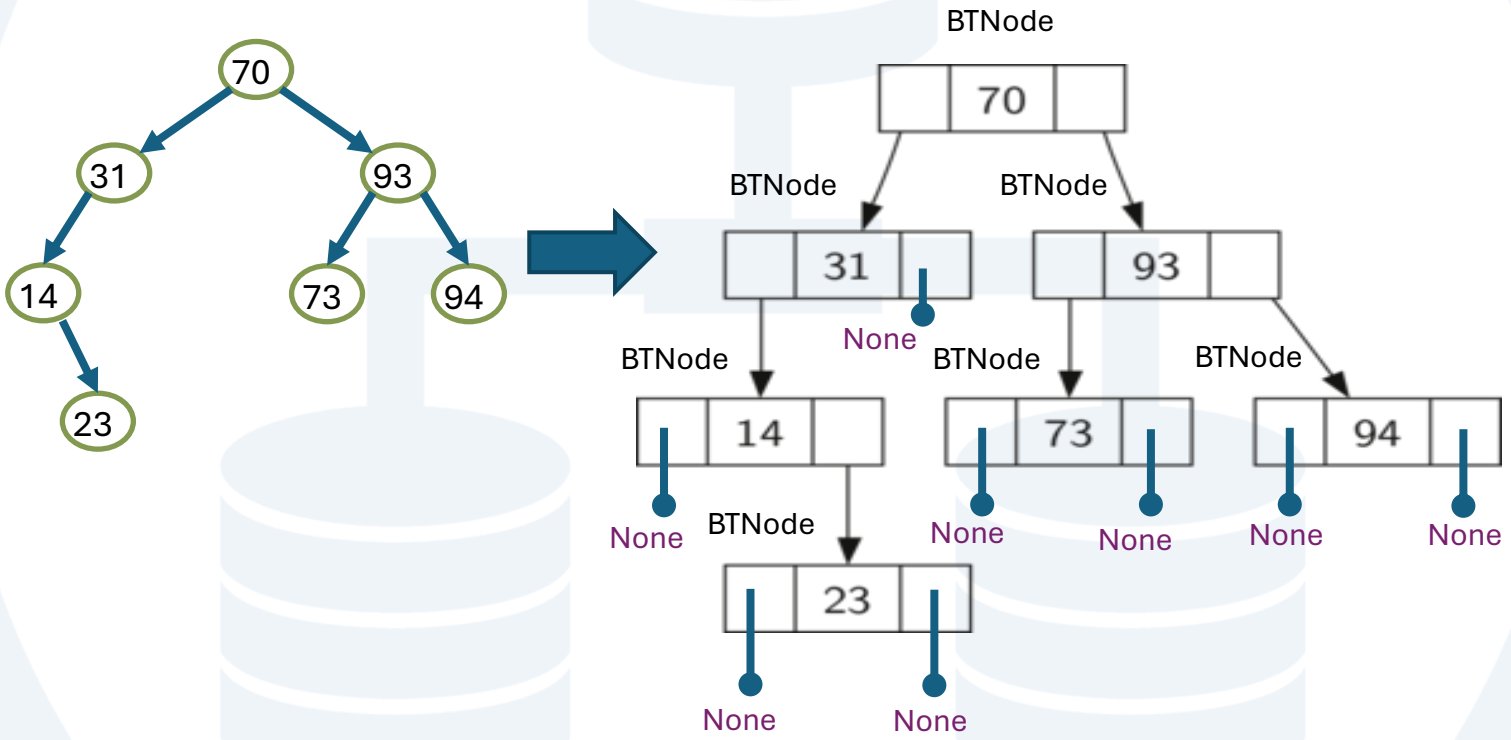
# Introduction to Binary Trees

- A binary tree is a tree in which each node has at most two children.
- The two children are referred to as the left child and right child.
- A node may have no child, one child, or two children.
- Each child can itself be the root of another binary tree (subtree).
- Binary trees provide a structured way to organise and process hierarchical data.
- Common operations include insertion, deletion, searching, and traversal

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Introduction to Binary Trees

Definition: Each node has at most two children.



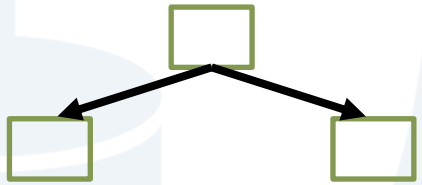

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Binary Tree Node Class

- Each node stores a **data value**.
- data stores the **value of the current node**.
- left stores a reference to the **left child**.
- right stores a reference to the **right child**.
- Initially, both child references are set to **None**.
- Child nodes can be connected later by assigning **Node objects** to left and right.

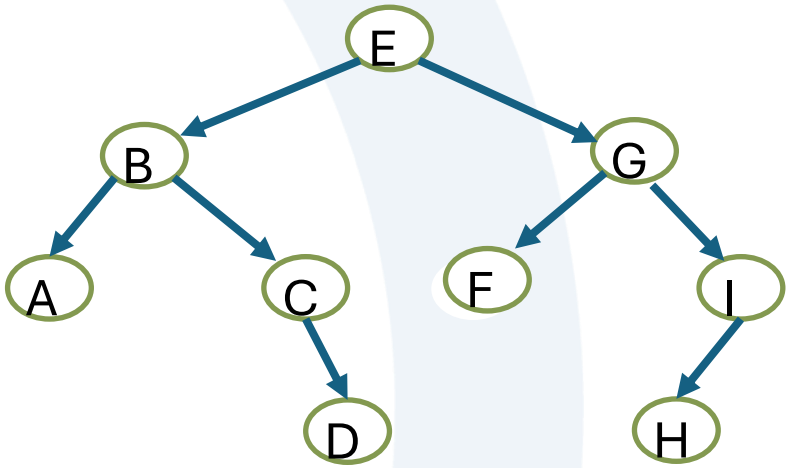
```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Binary Tree Operations

- A binary tree supports several fundamental operations:
  - **Insertion** – Add a new node to the tree.
  - **Deletion** – Remove an existing node from the tree.
  - **Searching** – Find a particular node or value.
  - **Traversal** – Visit every node systematically in a specific order.
- **Common Traversal Orders**
  - **Pre-order** – Root → Left → Right
  - **In-order** – Left → Root → Right
  - **Post-order** – Left → Right → Root
  - **Level-order** – Visit nodes level by level




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

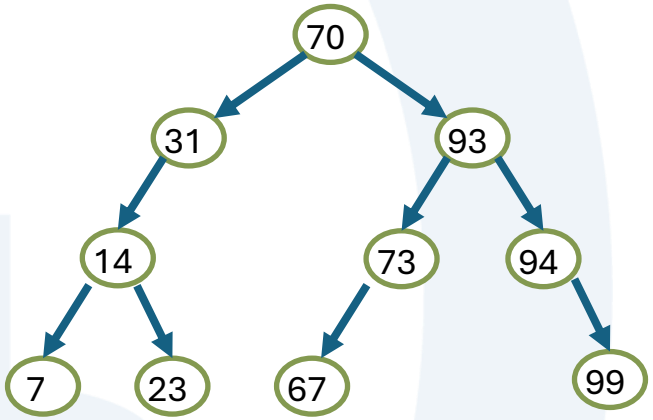
# Binary Tree Traversal

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Binary Tree Traversal

- **Tree traversal** is the process of **systematically visiting every node in a tree**.
- **Why do we traverse a binary tree?**
  - **Display** all nodes in the tree
  - **Search** for a particular node
  - **Count** the number of nodes
  - **Process** or examine node values

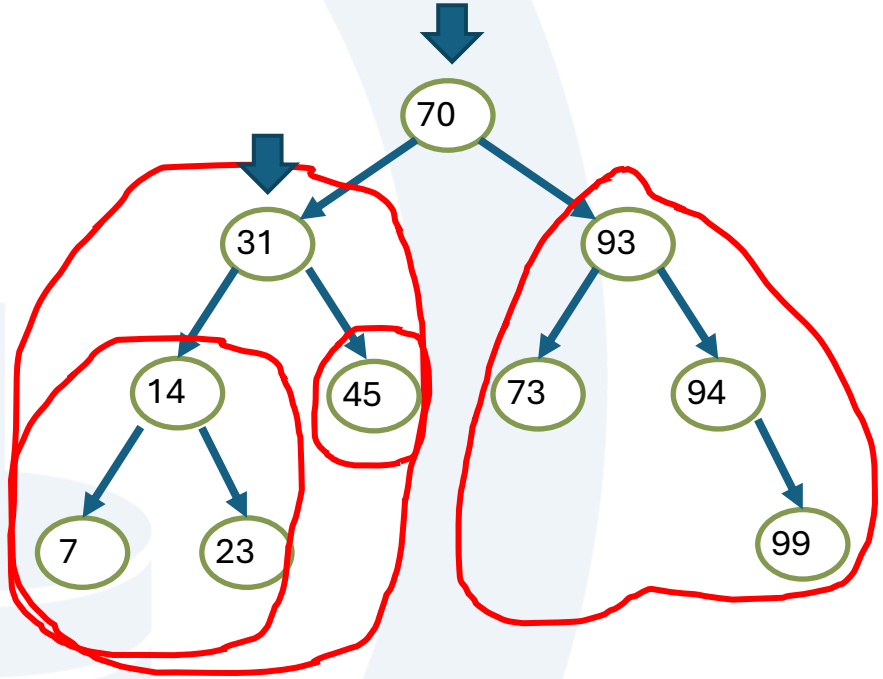



- **Two Main Traversal Approaches**
  - **Depth-First Traversal (DFS)** – explores a subtree deeply before moving to another subtree.
  - **Breadth-First Traversal (BFS)** – visits nodes **level by level**.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Binary Tree Traversal – Depth-first Traversals

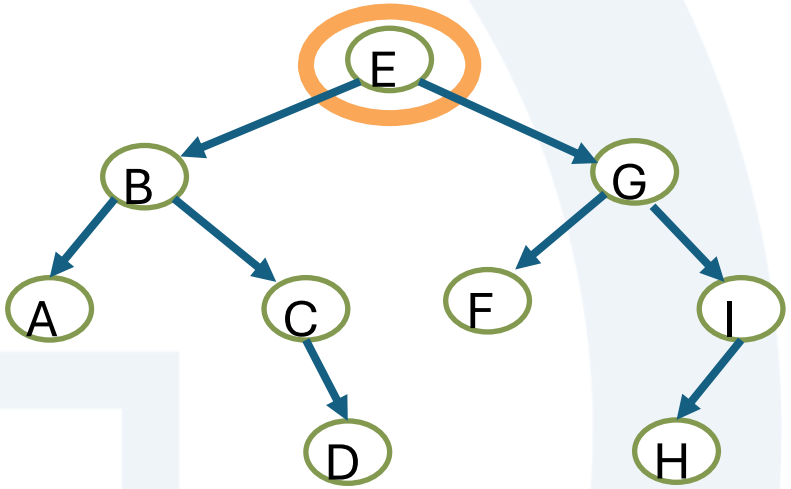
- Start from the **root** of the tree.
- Explore **one subtree deeply** before moving to another subtree.
- Treat each subtree as a **smaller tree** and apply the same traversal process.
- Continue until reaching a node with **no further child to explore**.
- Return to the previous node and continue with the **remaining subtree**.
- Depth-first traversal can be implemented using **recursion or a stack**.




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Pre-order Depth First Traversal

- Pre-order
  - **Process the current node's data**
  - **Visit the left child subtree**
  - **Visit the right child subtree**



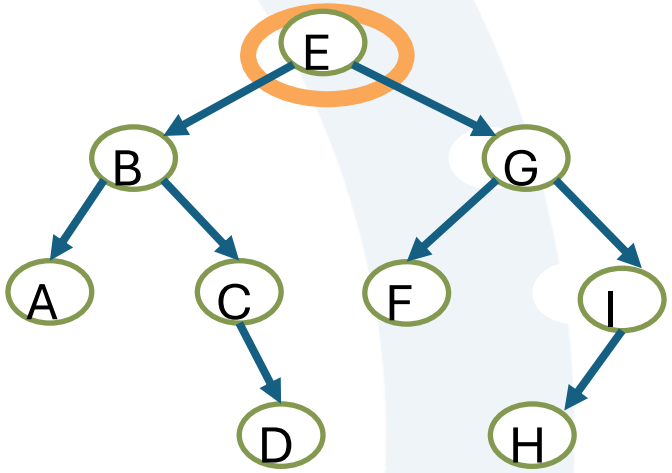

```
TreeTraversal(Node N):
    Visit N;
    If (N has left child)
        TreeTraversal(LeftChild);
    If (N has right child)
        TreeTraversal(RightChild);
    Return; // return to parent
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Pre-order Depth First Traversal

Output:

```
E B A C D G F I H
```

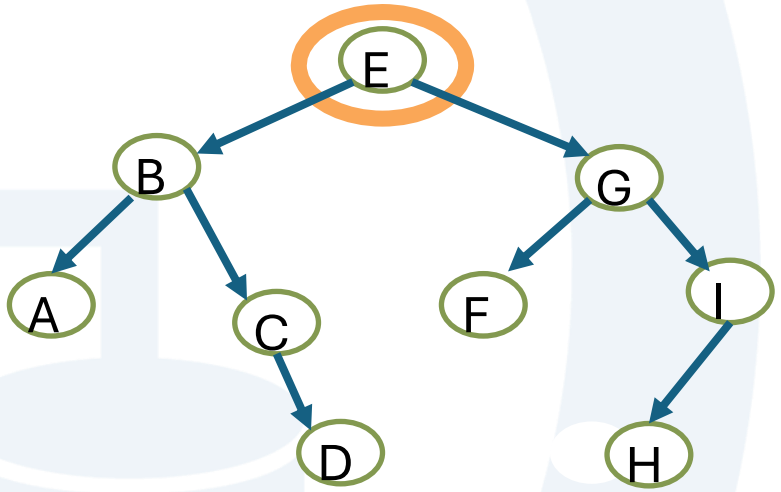



```
def pre_order_traversal(self, node):
        if node:
            print(node.data, end=" ")
            self.pre_order_traversal(node.left)
            self.pre_order_traversal(node.right)
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# In-order Depth First Traversal

- Pre-order
  - Process the current node's data
  - Visit the left child subtree
  - Visit the right child subtree
- **In-order**
  - **Visit the left child subtree**
  - **Process the current node's data**
  - **Visit the right child subtree**
- Post-order

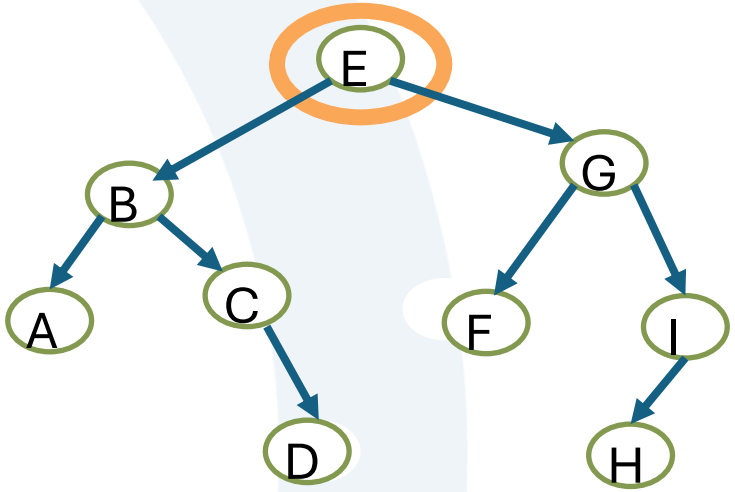



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# In-order Depth First Traversal

Output:

```
A B C D E F G H I
```

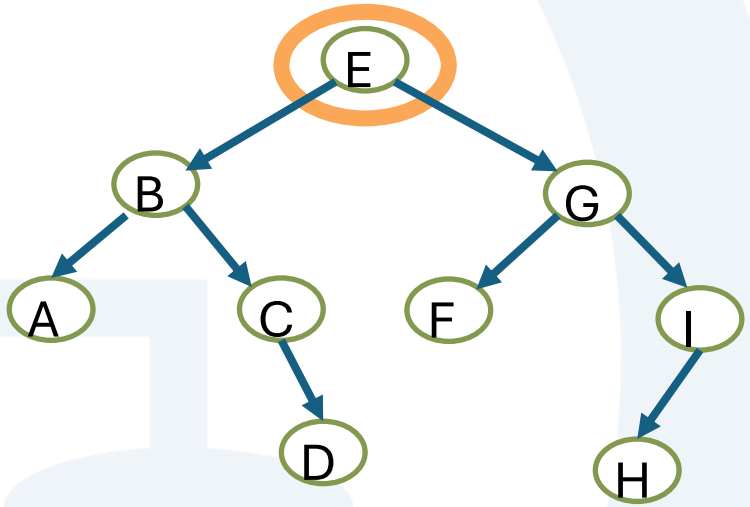



```
def in_order_traversal(self, node):
        if node:
            self.in_order_traversal(node.left)
            print(node.data, end=" ")
            self.in_order_traversal(node.right)
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Post-order Depth First Traversal

- Pre-order
  - Process the current node's data
  - Visit the left child subtree
  - Visit the right child subtree
- In-order
  - Visit the left child subtree
  - Process the current node's data
  - Visit the right child subtree
- **Post-order**
  - **Visit the left child subtree**
  - **Visit the right child subtree**
  - **Process the current node's data**

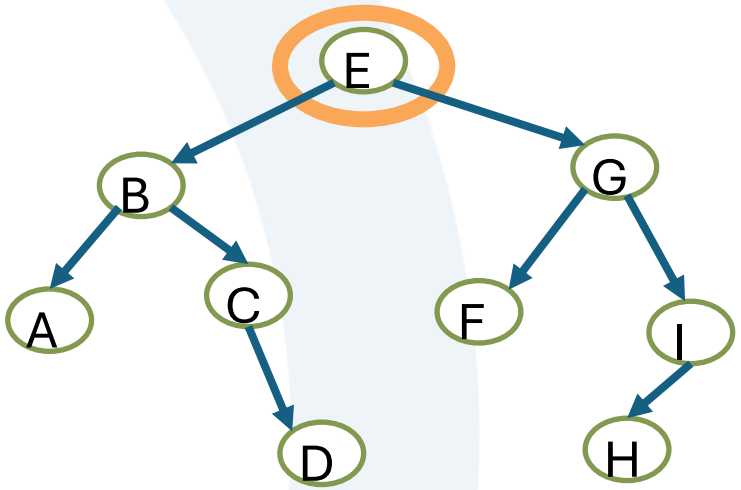



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Post-order Depth First Traversal

Output:

```
A D C B F H I G E
```




```
def post_order_traversal(self, node):
        if node:
            self.post_order_traversal(node.left)
            self.post_order_traversal(node.right)
            print(node.data, end=" ")
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Summary of Depth First Traversal

Pre-Order Traversal

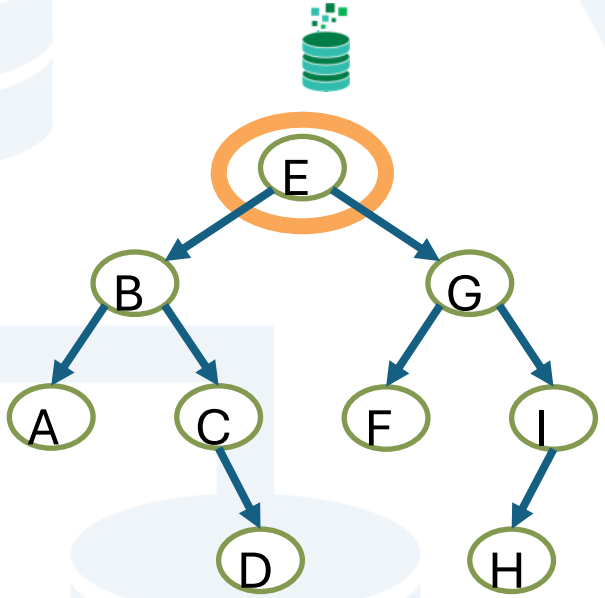
```
E B A C D G F I H
```

In-Order Traversal

```
A B C D E F G H I
```

Post-Order Traversal

```
A D C B F H I G E
```




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Breadth-first Traversal: Level-by-level

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Breadth-first Traversal: Level-by-level

**Level-By-Level Traversal:**

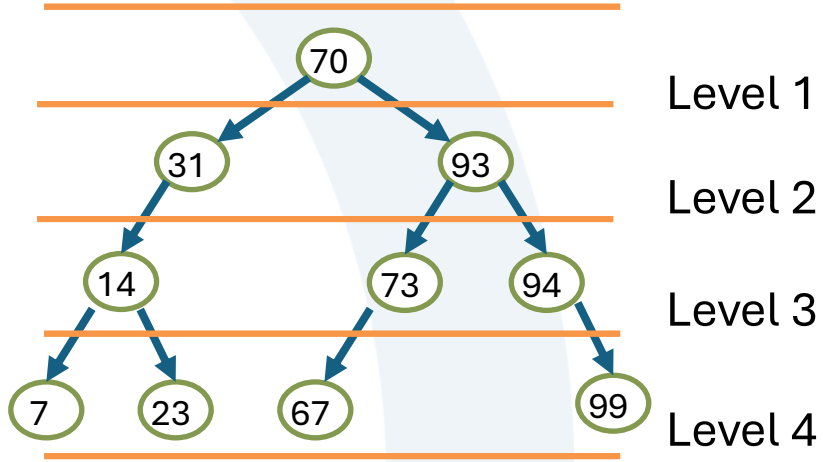
**Visit the root (Level 1)**

**Visit children of the root (Level 2)**

**Visit grandchildren of the root (Level3)** …

How?

- Visiting a node
- Remember all its children
  - Use a queue (FIFO structure)




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

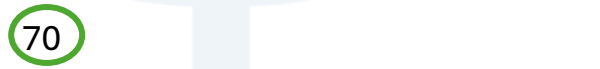
# Breadth-first Traversal: Level-by-level

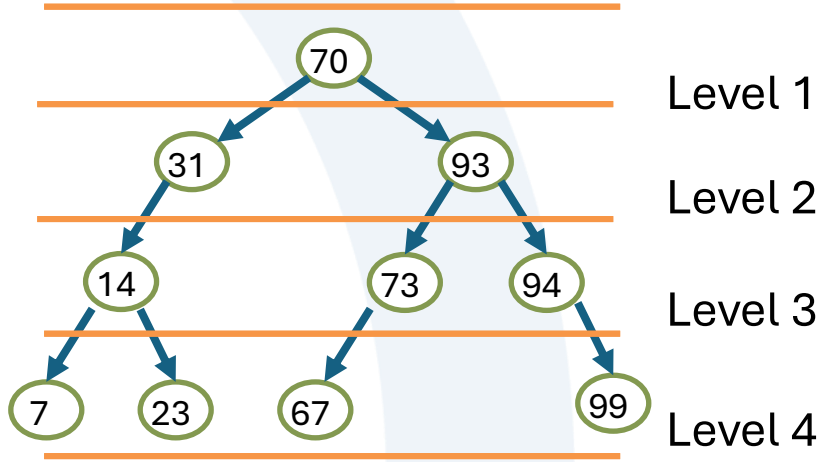
**Level-By-Level Traversal:**

**Visit the root (Level 1)**

**Visit children of the root (Level 2)**

**Visit grandchildren of the root (Level3)** ...




1. Enqueue the current node
2. Dequeue a node
3. Enqueue the left and right children if they exist
4. Repeat Step 2 until the queue is empty

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Breadth-first Traversal: Level-by-level

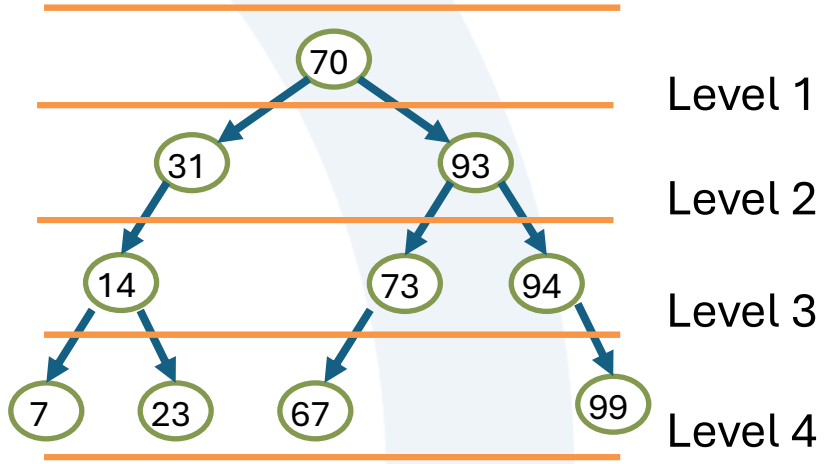
**Level-By-Level Traversal:**

**Visit the root (Level 1)**

**Visit children of the root (Level 2)**

**Visit grandchildren of the root (Level3)** ...




1. Enqueue the current node
2. Dequeue a node
3. Enqueue the left and right children if they exist
4. Repeat Step 2 until the queue is empty

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Breadth-first Traversal: Level-by-level

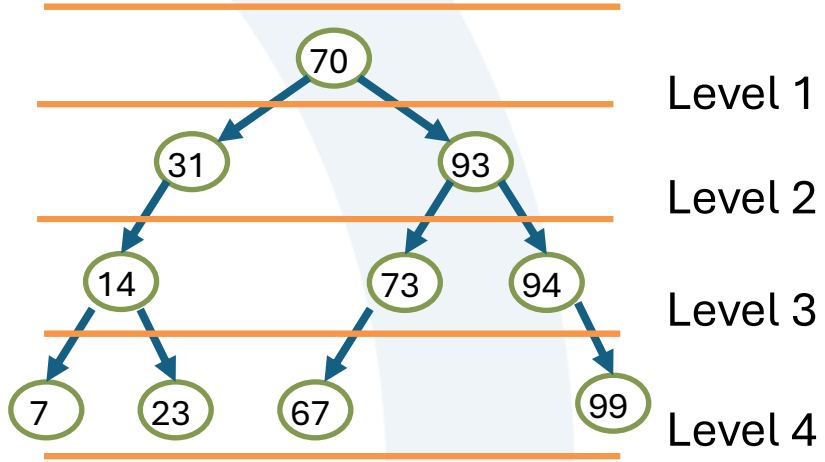
**Level-By-Level Traversal:**

**Visit the root (Level 1)**

**Visit children of the root (Level 2)**

**Visit grandchildren of the root (Level3)** ...




1. Enqueue the current node
2. Dequeue a node
3. Enqueue the left and right children if they exist
4. Repeat Step 2 until the queue is empty

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

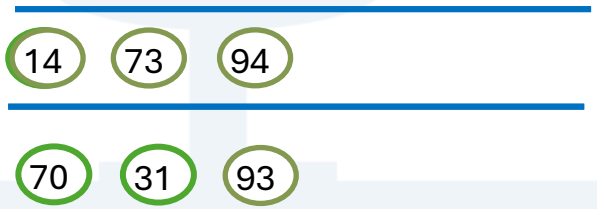
# Breadth-first Traversal: Level-by-level

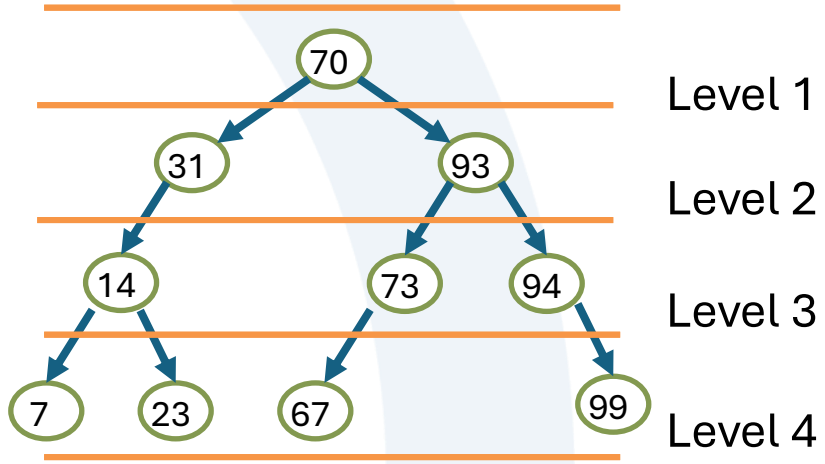
**Level-By-Level Traversal:**

**Visit the root (Level 1)**

**Visit children of the root (Level 2)**

**Visit grandchildren of the root (Level3)** ...




1. Enqueue the current node
2. Dequeue a node
3. Enqueue the left and right children if they exist
4. Repeat Step 2 until the queue is empty

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

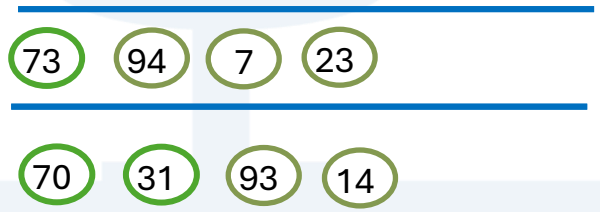
# Breadth-first Traversal: Level-by-level

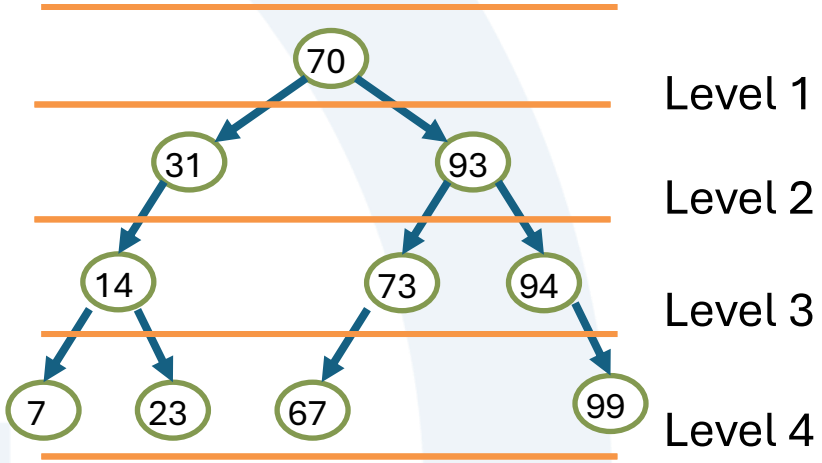
**Level-By-Level Traversal:**

**Visit the root (Level 1)**

**Visit children of the root (Level 2)**

**Visit grandchildren of the root (Level3)** ...



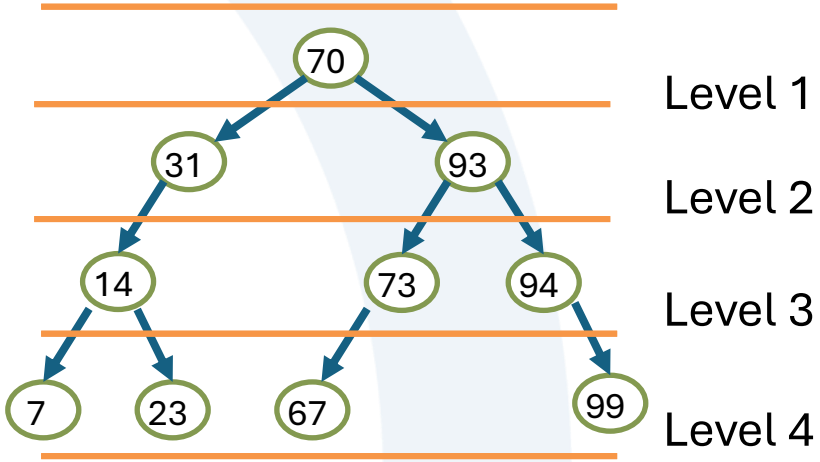

1. Enqueue the current node
2. Dequeue a node
3. Enqueue the left and right children if they exist
4. Repeat Step 2 until the queue is empty

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Breadth-first Traversal: Level-by-level

Level-By-Level Traversal:

1. Enqueue the current node
2. Dequeue a node
3. Enqueue the left and right children if they exist
4. Repeat Step 2 until the queue is empty



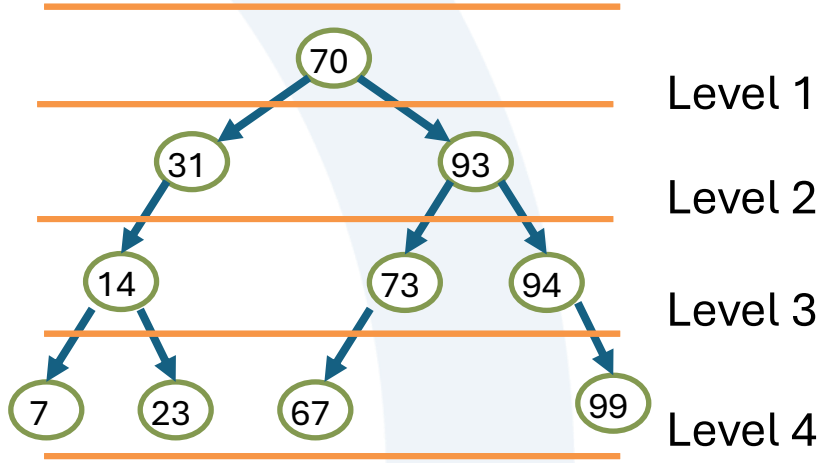




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Breadth-first Traversal: Level-by-level

1. **Initialize the queue:**
   - Create an instance of the Queue class.
   - Add the root node to the queue using **queue.enqueue(self.root).**
2. **Start the traversal loop:**
   - While the queue is not empty:
     - Dequeue the front node using **queue.dequeue()** and print its data.
     - Enqueue the left child if it exists.
     - Enqueue the right child if it exists.
3. **Finish traversal:**
   - Repeat the process until the queue is empty, ensuring level-by-level traversal.




```
def level_order_traversal(self):
        if not self.root:
            return

        queue = Queue()
        queue.enqueue(self.root)

        while not queue.is_empty():
            current_node = queue.dequeue()
            print(current_node.data, end=" ")

            if current_node.left:
                    queue.enqueue(current_node.left)
            if current_node.right:
                    queue.enqueue(current_node.right)
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

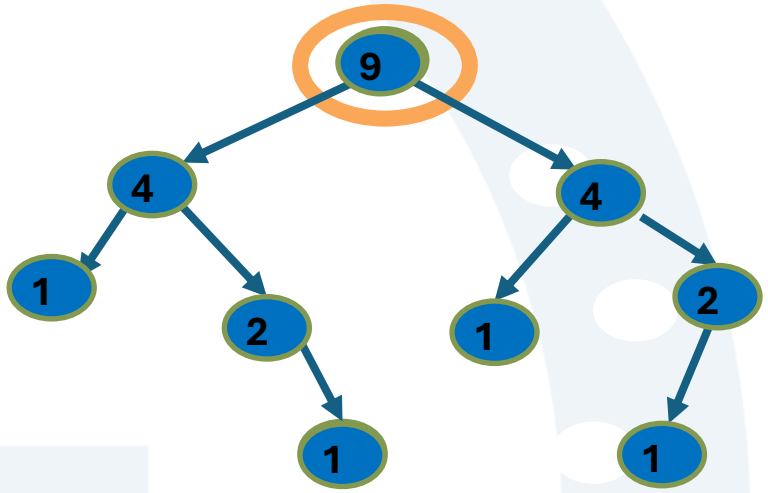
# Binary Tree Applications

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Count Nodes in a Binary Tree (SIZE)

- Recursive definition:
  - Number of nodes in a tree
    = **1**
    + **number of nodes in left subtree**
    + **number of nodes in right subtree**
- Each node returns the number of nodes in its subtree

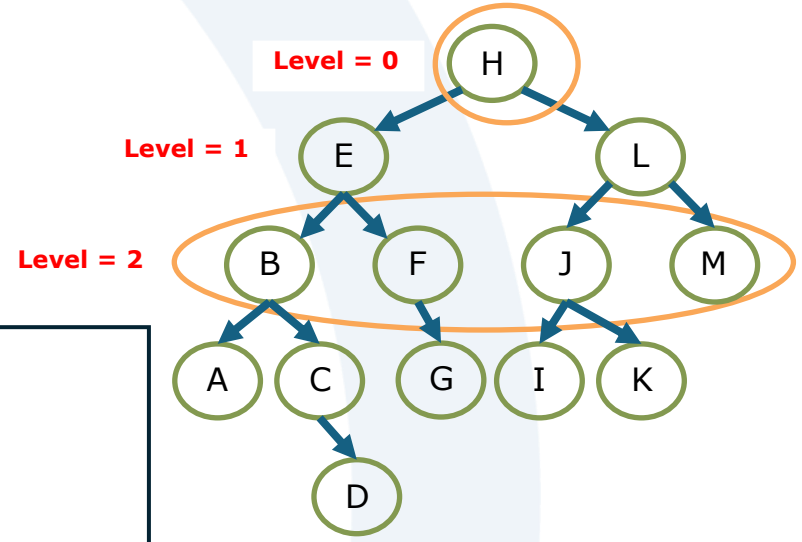



```
def count_nodes(self, node):
    if not node:
        return 0
    return 1 + self.count_nodes(node.left) + self.count_nodes(node.right)
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

# Find the Kth-level Descendant Nodes

- Given a node X, find all the descendants of X
- Given node **E** and **K=2**, we should return its grandchild nodes A, C, and G
- What if we want to find $K^{th}$-level descendants?
  - **Need a way to keep track of how many levels down we've gone**




```
def kth_level_descendants(self, node, k): # k =2
    if node is None:
        return []
    if k == 0:
        return [node.data]
    left_descendants = self.kth_level_descendants(node.left, k - 1)
    right_descendants = self.kth_level_descendants(node.right, k - 1)

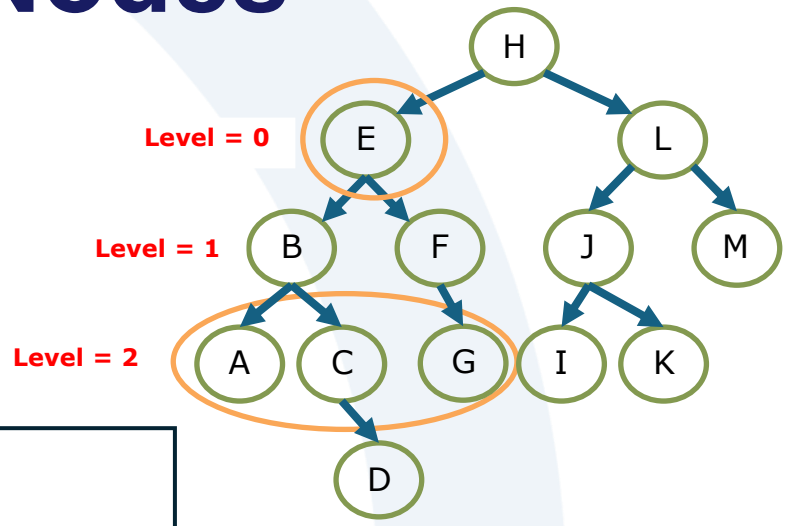
    return left_descendants + right_descendants
```

```
print(bt.kth_level_descendants(bt.root, 2))
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

# Find the Kth-level Descendant Nodes

- Given a node X, find all the descendants of X
- Given node **E** and **K=2**, we should return its grandchild nodes A, C, and G
- What if we want to find $K^{th}$-level descendants?
  - **Need a way to keep track of how many levels down we've gone**




```
def kth_level_descendants(self, node, k): # k =2
    if node is None:
        return []
    if k == 0:
        return [node.data]
    left_descendants = self.kth_level_descendants(node.left, k - 1)
    right_descendants = self.kth_level_descendants(node.right, k - 1)

    return left_descendants + right_descendants
```

```
print(bt.kth_level_descendants(E, 2))
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

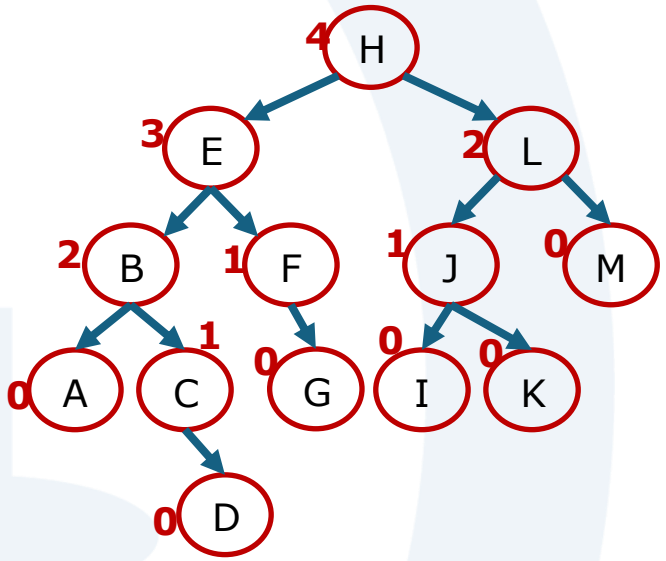
# Height of A Node in A Binary Tree

- The height of a tree: The number of edges on the longest path from the root to a leaf
  - Leaf node **returns 0**
  - None node (empty) **returns ?**

```
def calculate_height(self, node):
        if node is None:
            return ?

        left_height = self.calculate_height(node.left)
        right_height = self.calculate_height(node.right)

        return 1 + max(left_height, right_height)
```




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

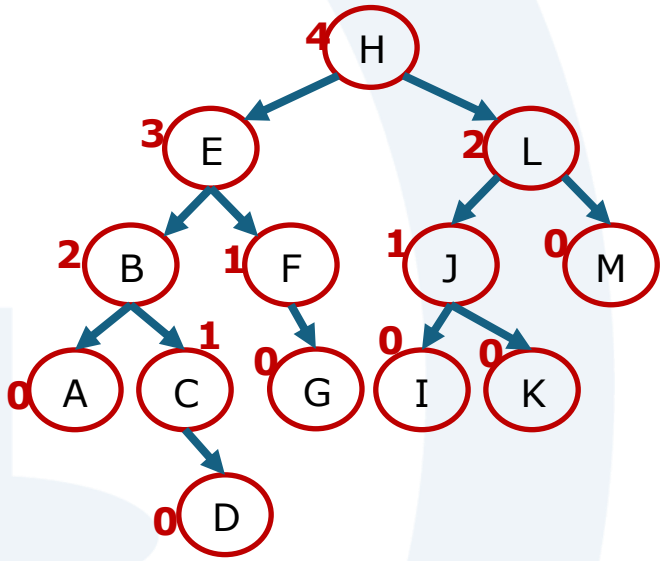
# Height of A Node in A Binary Tree

- The height of a tree: The number of edges on the longest path from the root to a leaf
  - Leaf node **returns 0**
  - None node (empty) **returns -1**

```
def calculate_height(self, node):
        if node is None:
            return -1

        left_height = self.calculate_height(node.left)
        right_height = self.calculate_height(node.right)

        return 1 + max(left_height, right_height)
```




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

# Pre-order Traversal with Stack

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

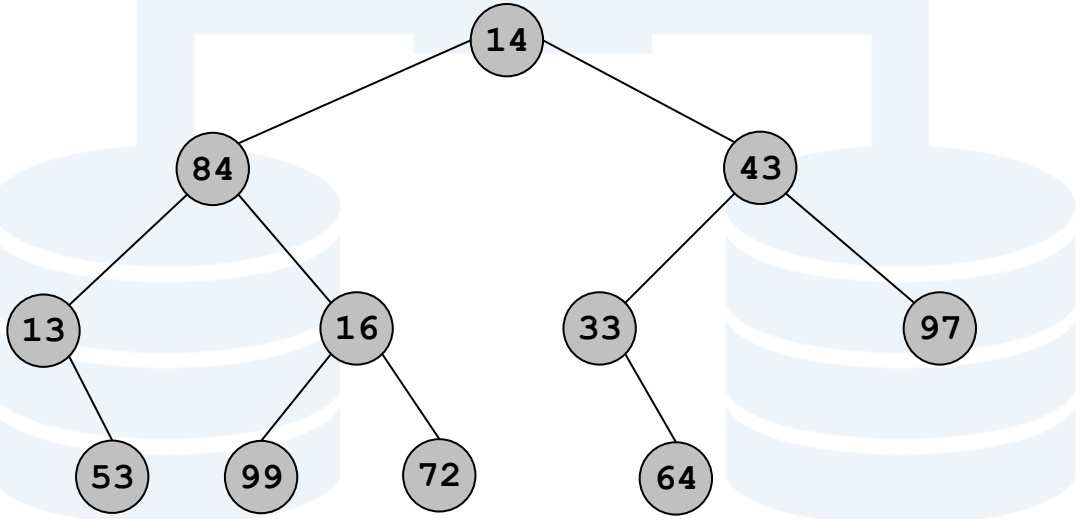
**Push the root onto the stack.**

**While the stack is not empty:**

Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

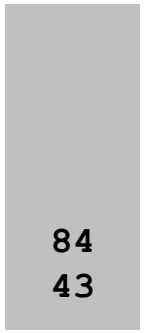
**Push the root onto the stack.**

**While the stack is not empty:**

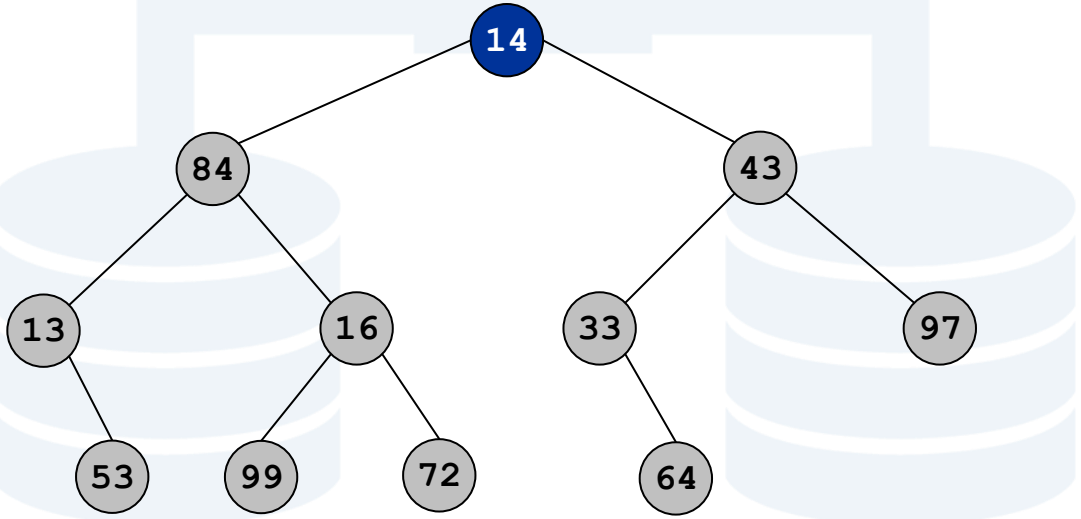
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

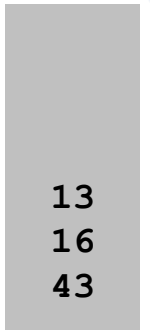
**Push the root onto the stack.**

**While the stack is not empty:**

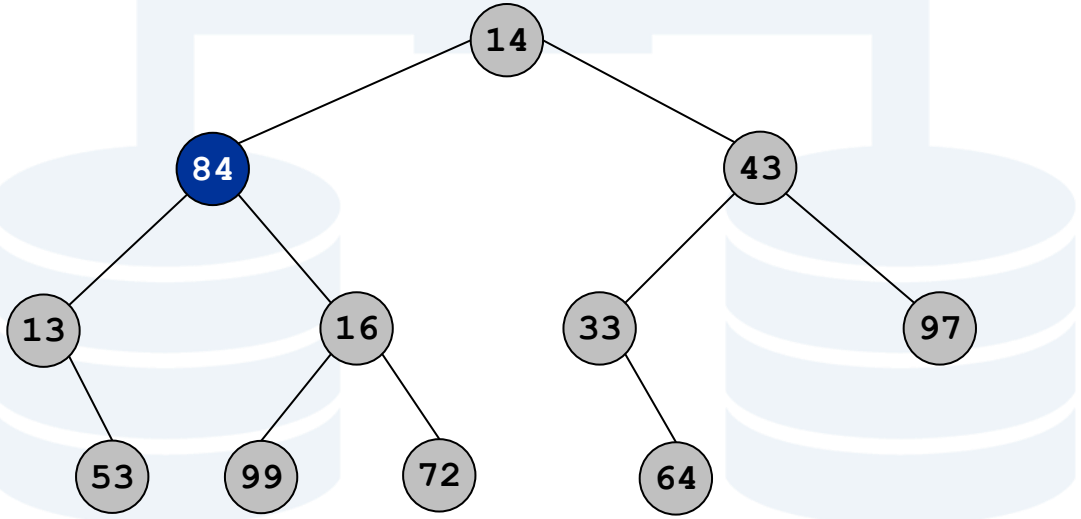
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

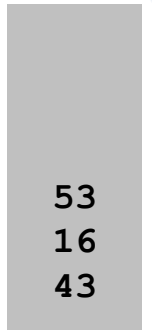
**Push the root onto the stack.**

**While the stack is not empty:**

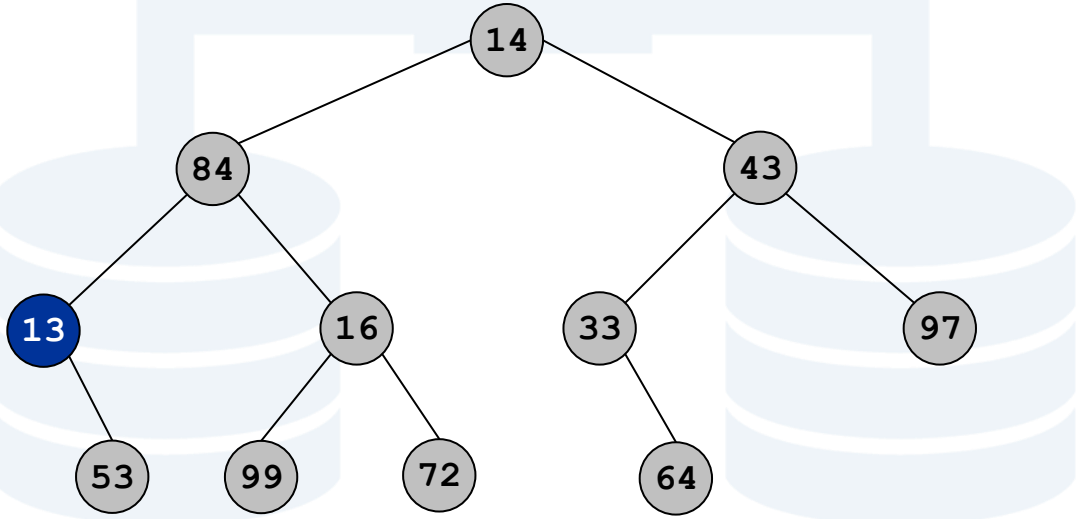
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

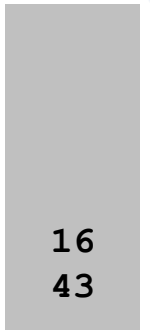
**Push the root onto the stack.**

**While the stack is not empty:**

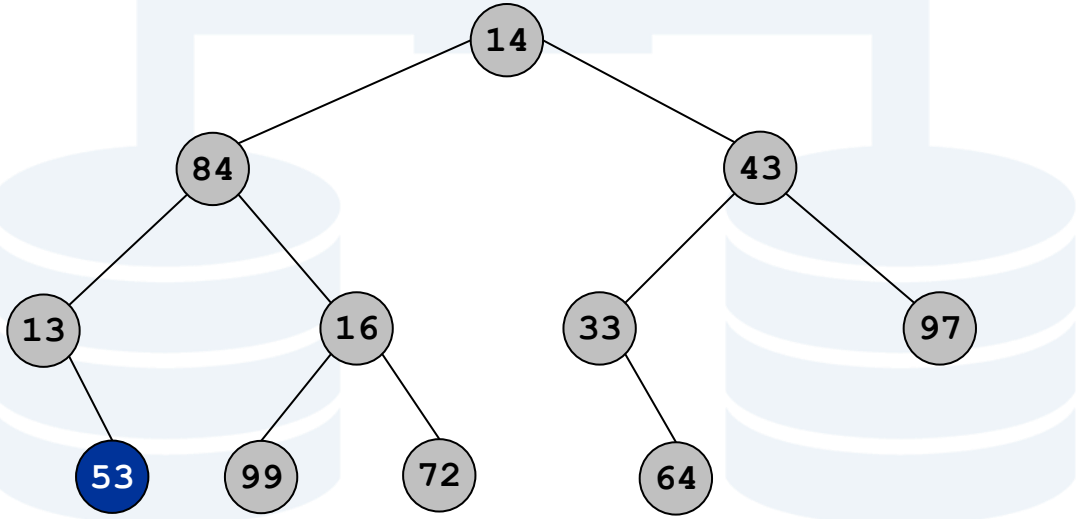
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

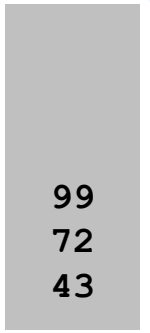
**Push the root onto the stack.**

**While the stack is not empty:**

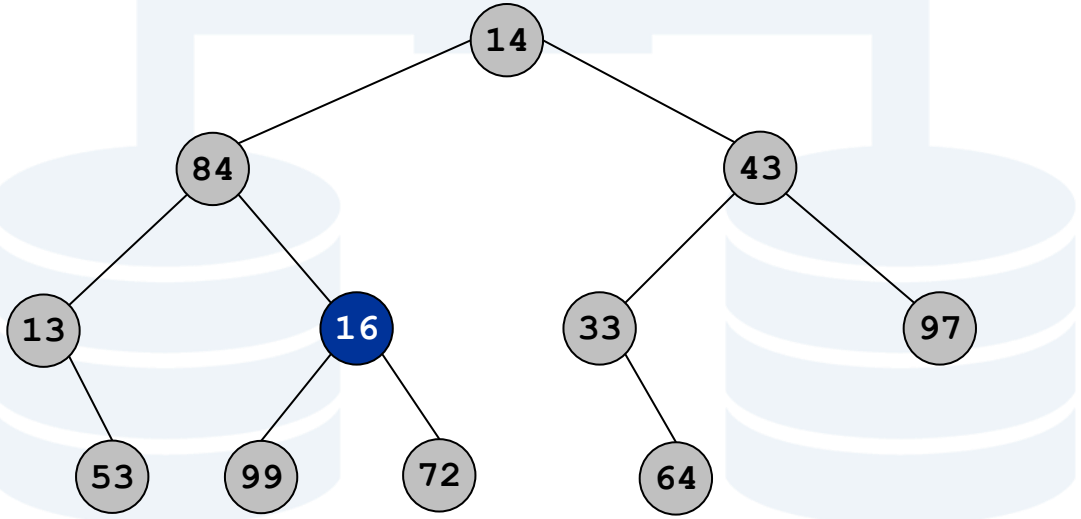
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

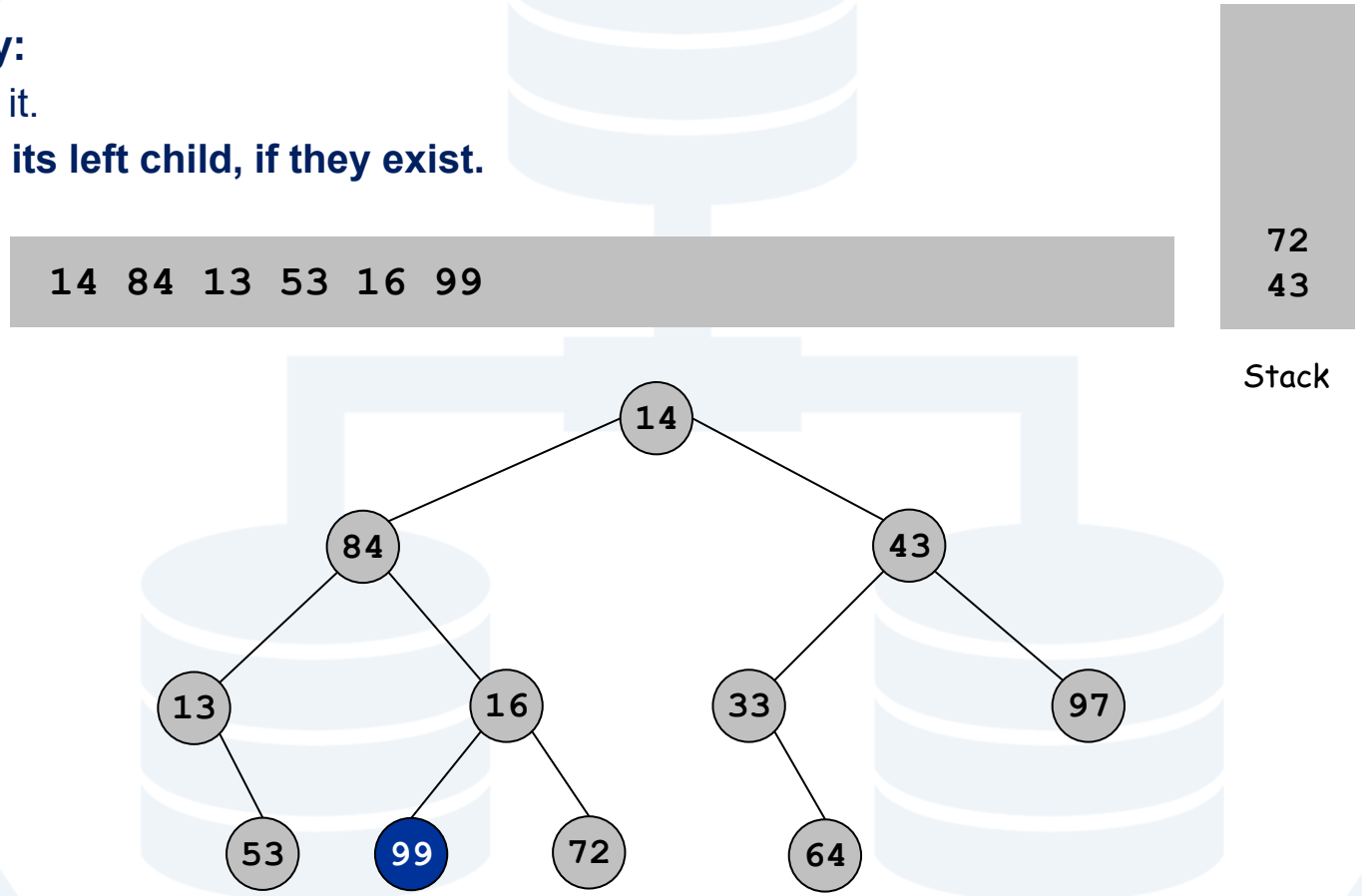
# Pre-order Traversal with Stack

**Push the root onto the stack.**

**While the stack is not empty:**

Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

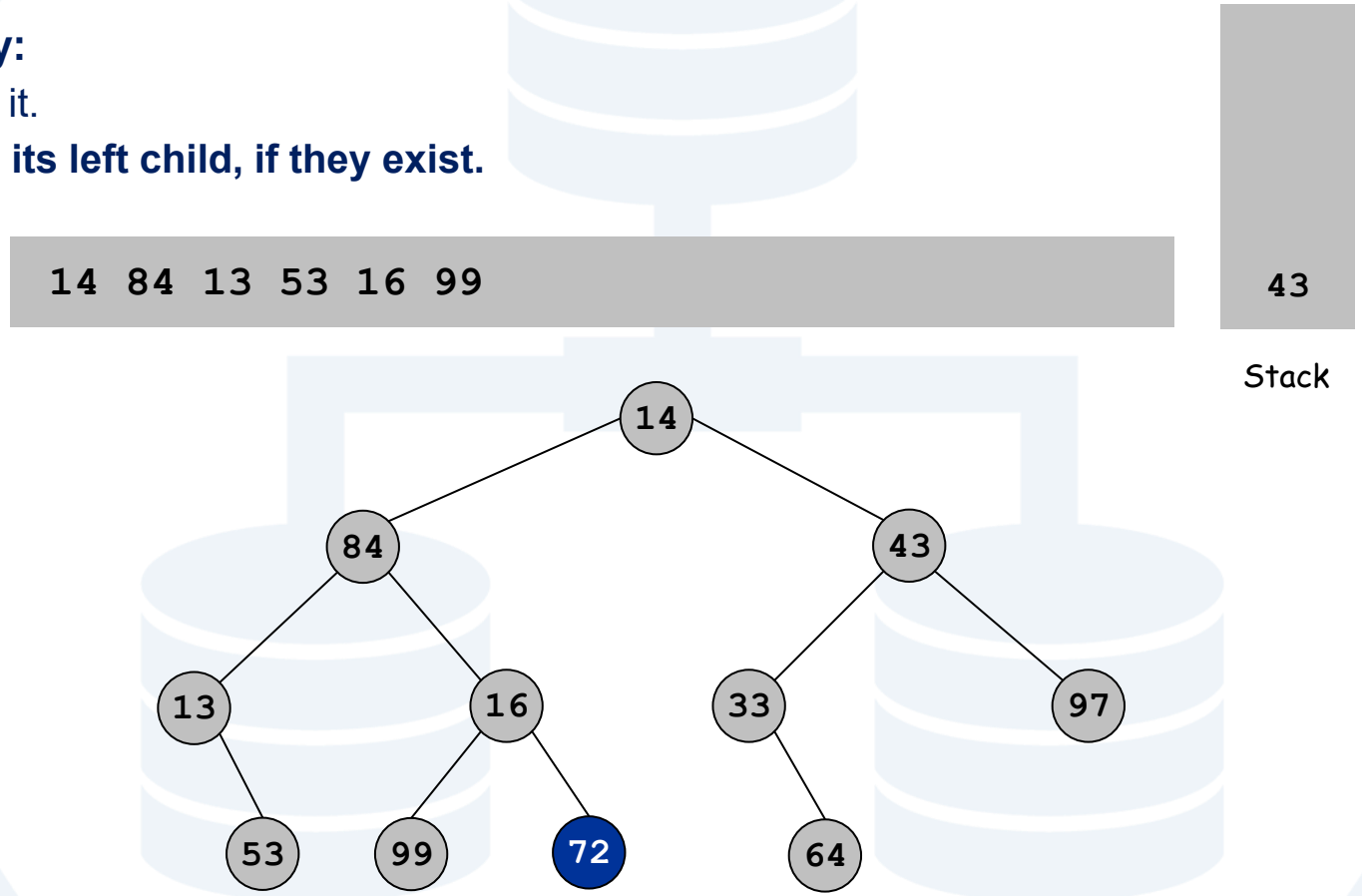
# Pre-order Traversal with Stack

**Push the root onto the stack.**

**While the stack is not empty:**

Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

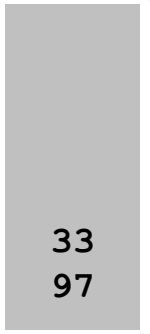
**Push the root onto the stack.**

**While the stack is not empty:**

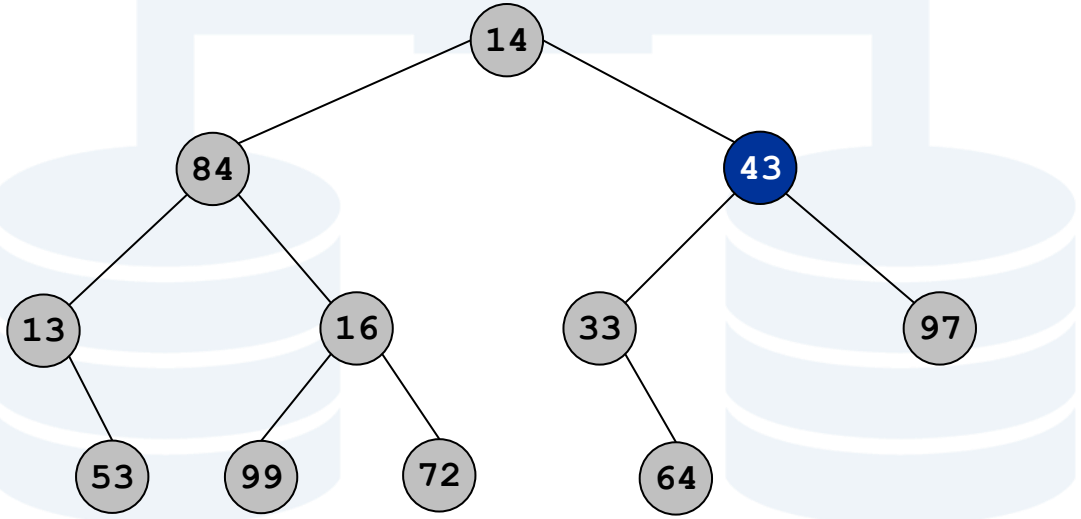
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

**Push the root onto the stack.**

**While the stack is not empty:**

Pop the top node and visit it.

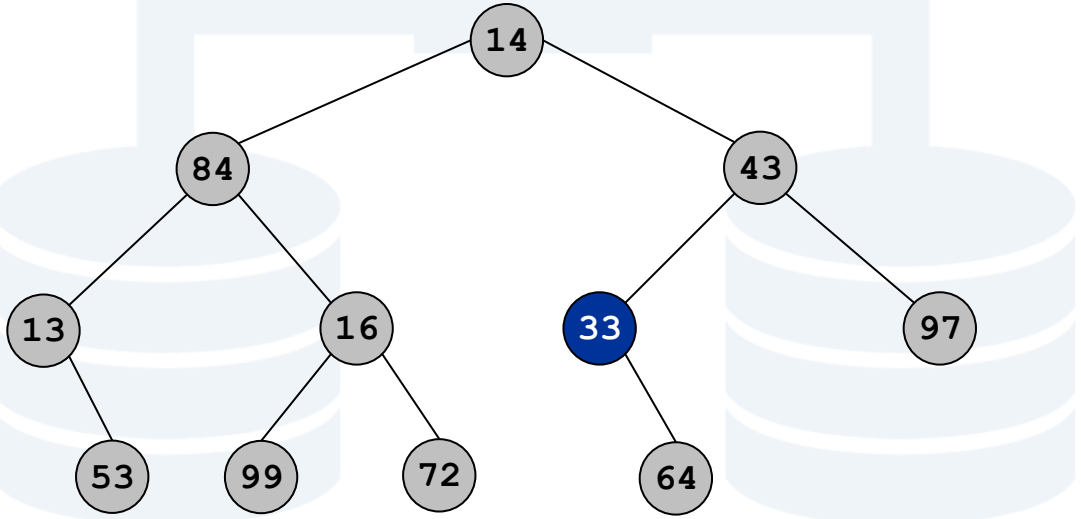
**Push its right child, then its left child, if they exist.**







Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

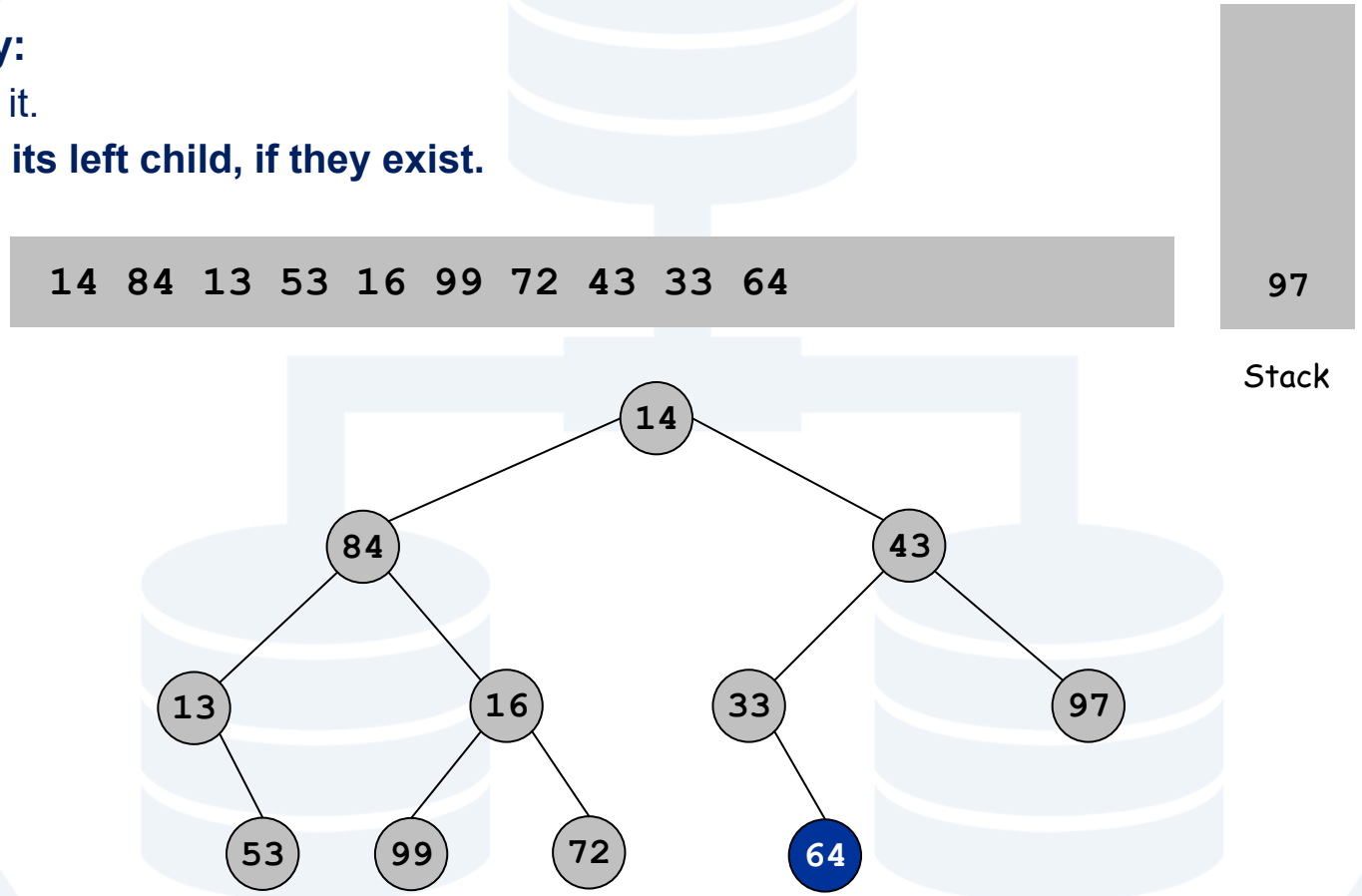
# Pre-order Traversal with Stack

**Push the root onto the stack.**

**While the stack is not empty:**

Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Pre-order Traversal with Stack

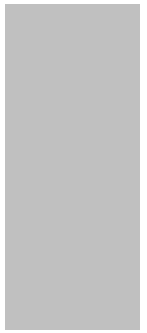
**Push the root onto the stack.**

**While the stack is not empty:**

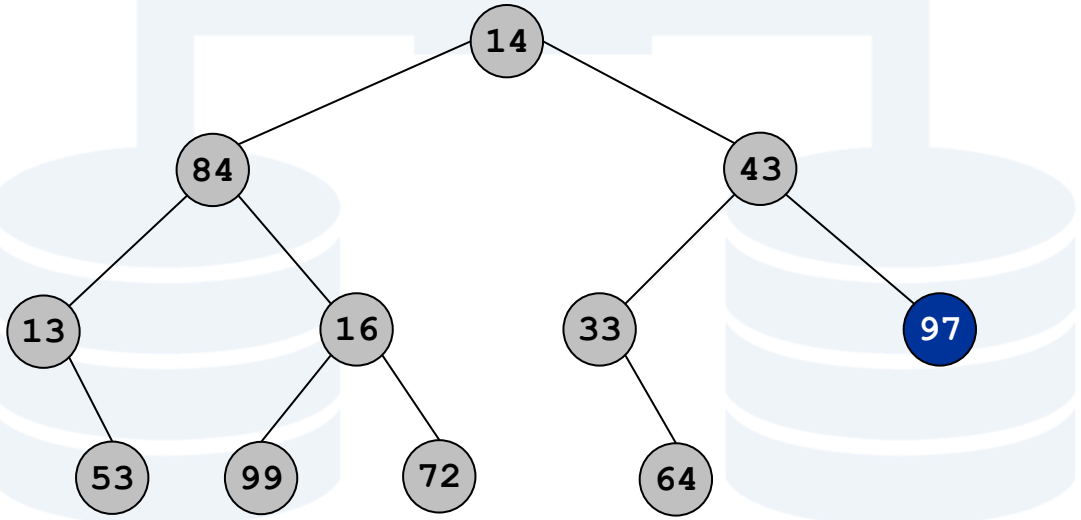
Pop the top node and visit it.

**Push its right child, then its left child, if they exist.**



Stack




© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Conclusion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

# Conclusion



- Trees organise data using **hierarchical relationships**.
- A binary tree allows each node to have **at most two children**.
- Binary tree nodes contain **data, left, and right references**.
- **Depth-first traversal** includes pre-order, in-order, and post-order.
- **Breadth-first traversal** visits nodes **level by level using a queue**.
- **Recursion, stacks, and queues** support efficient tree processing.
- Tree techniques can solve problems such as **counting nodes, finding descendants, and calculating height**.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished or transmitted in any form or by any means without written approval from the University.